Java Coding Interview Question Bank

Deduplicated, ranked by importance and grouped by topic
Prepared for a Test Automation Engineer with 4+ years of experience

Questions in your original document	**304**
Exact duplicates removed (same question, different punctuation, case or formatting)	**9**
Near-duplicates merged (same question asked in different words)	**144**
Unique questions in this PDF	**151**

Section 1: Critical (Must Practise)	27 questions
Section 2: High Priority	63 questions
Section 3: Medium Priority	45 questions
Section 4: Low Priority / Quick Revision	16 questions

How to Use This Document

Programs are ordered by how likely they are to be asked and how much weight interviewers give them for a tester with 4+ years of experience. Work top to bottom: finish Section 1 before you spend time on Section 3 or 4. Inside each section the questions are grouped by topic, and within each topic the most important question comes first.

A red tag such as **[asked x3 in your bank]** means the same question showed up several times in your original document (in different wording). Repetition is a good signal that it is a frequently asked question.

Section	What it means
Critical (Must Practise) 27 questions	Asked in almost every coding round. Write each one by hand, in at least two ways (with and without built-in methods), and be ready to state time complexity and handle edge cases.
High Priority 63 questions	Frequent follow-ups and the next level of difficulty. Practise these once you can do Section 1 without hesitation, with focus on HashMap, Streams and file handling.
Medium Priority 45 questions	Occasionally asked, usually for SDET-leaning roles or as a challenge question. Know the approach even if you cannot write it perfectly.
Low Priority / Quick Revision 16 questions	Rarely asked for automation testers. Skim the idea, do not spend prep time here before Sections 1 to 3.

What Interviewers Look For (4+ Years)

Topic	What the interviewer is testing
String Programs	The most common live-coding round. They watch your approach (with and without built-ins), edge cases (null, empty, case) and complexity.
Number & Math Programs	Warm-up logic: loops, recursion, and clean handling of boundaries. Should be answerable in minutes.
Array Programs	Can you solve with and without extra collections, and explain time vs space trade-offs?
Collections & HashMap	Practical use of Map/Set/List: frequency counts, de-duplication, sorting maps, test-data handling.
Java 8 Streams	Modern, concise code: filter, map, collect, distinct, sorted, grouping.
Sorting & Searching	Basic algorithm knowledge and the ability to reason about complexity.
OOP, Design Pattern & Exception	Singleton, equals/hashCode, comparators and custom exceptions written correctly.
File Handling	Reading, writing and parsing files, which is common in test-data utilities.
Multithreading	Odd/even thread printing and deadlock are the usual pair; shows if you understand thread coordination.
Linked List, Stack & Matrix	Data-structure questions, seldom asked for testers but common for SDET-style roles.
Output & Concept Tricks	Short 'what will this print / is this allowed' questions.

Note: many entries were the same program phrased as 'Write a java code for...', 'Java program to...' or 'WAP to...'; they are merged into one task. One resume line ('Created a collection of employee records and sorted them by employee ID') was turned into a task. The 'Coding tasks:' prefix was dropped.

Table of Contents

Section 1: Critical (Must Practise) (27 questions) . . . 5
String Programs (10) . . . 5
Number & Math Programs (5) . . . 7
Array Programs (6) . . . 8
Collections & HashMap Programs (incl. Test Data Handling) (5) . . . 9
Sorting, Searching & Complexity (1) . . . 10

Section 2: High Priority (63 questions) . . . 11
String Programs (14) . . . 11
Number & Math Programs (11) . . . 14
Array Programs (13) . . . 15
Collections & HashMap Programs (incl. Test Data Handling) (8) . . . 18
Java 8 Streams Programs (5) . . . 19
Sorting, Searching & Complexity (2) . . . 20
OOP, Design Pattern & Exception Coding (7) . . . 20
File Handling Programs (1) . . . 22
Multithreading Programs (2) . . . 22

Section 3: Medium Priority (45 questions) . . . 23
String Programs (8) . . . 23
Number & Math Programs (4) . . . 24
Array Programs (7) . . . 25
Collections & HashMap Programs (incl. Test Data Handling) (4) . . . 27
Java 8 Streams Programs (3) . . . 27
Sorting, Searching & Complexity (2) . . . 28
OOP, Design Pattern & Exception Coding (1) . . . 29
File Handling Programs (4) . . . 29
Multithreading Programs (2) . . . 30
Linked List, Stack, Matrix & Advanced Problems (8) . . . 30
Output & Concept Trick Questions (2) . . . 32

Section 4: Low Priority / Quick Revision (16 questions) . . . 33
String Programs (4) . . . 33
Number & Math Programs (5) . . . 33
Array Programs (4) . . . 34
Sorting, Searching & Complexity (1) . . . 35
Linked List, Stack, Matrix & Advanced Problems (2) . . . 36

Topic Finder (all sections for one topic)

Use this if you want to study one topic at a time. Each number links to that topic's questions inside a section.

Topic	S1	S2	S3	S4	Total
String Programs	10 Qs, p.5	14 Qs, p.11	8 Qs, p.23	4 Qs, p.33	**36**
Number & Math Programs	5 Qs, p.7	11 Qs, p.14	4 Qs, p.24	5 Qs, p.33	**25**
Array Programs	6 Qs, p.8	13 Qs, p.15	7 Qs, p.25	4 Qs, p.34	**30**
Collections & HashMap Programs (incl. Test Data Handling)	5 Qs, p.9	8 Qs, p.18	4 Qs, p.27	-	**17**
Java 8 Streams Programs	-	5 Qs, p.19	3 Qs, p.27	-	**8**
Sorting, Searching & Complexity	1 Qs, p.10	2 Qs, p.20	2 Qs, p.28	1 Qs, p.35	**6**
OOP, Design Pattern & Exception Coding	-	7 Qs, p.20	1 Qs, p.29	-	**8**
File Handling Programs	-	1 Qs, p.22	4 Qs, p.29	-	**5**
Multithreading Programs	-	2 Qs, p.22	2 Qs, p.30	-	**4**
Linked List, Stack, Matrix & Advanced Problems	-	-	8 Qs, p.30	2 Qs, p.36	**10**
Output & Concept Trick Questions	-	-	2 Qs, p.32	-	**2**

S1 = Critical, S2 = High, S3 = Medium, S4 = Low priority.

Section 1: Critical (Must Practise) (27 questions)

Asked in almost every coding round. Write each one by hand, in at least two ways (with and without built-in methods), and be ready to state time complexity and handle edge cases.

String Programs (10)

1. Reverse a string (with and without built-in methods, using StringBuilder or a loop). **[asked x10 in your bank]**

   Answer: Loop from the last character to the first, appending to a StringBuilder (O(n)). The built-in way is new StringBuilder(s).reverse().

   Example: Reverse a product code like "SKU-4521" to validate a check pattern.

```
static String reverse(String s) {
  StringBuilder sb = new StringBuilder();
  for (int i = s.length() - 1; i >= 0; i--) sb.append(s.charAt(i));
  return sb.toString();   // built-in: new StringBuilder(s).reverse().toString()
}
```

2. Check whether a string is a palindrome. **[asked x6 in your bank]**

   Answer: Compare characters from both ends moving inward; any mismatch means not a palindrome (O(n), O(1) space).

   Example: Check whether a generated coupon code reads the same both ways.

```
static boolean isPalindrome(String s) {
  int i = 0, j = s.length() - 1;
  while (i < j) if (s.charAt(i++) != s.charAt(j--)) return false;
  return true;
}
```

3. Check whether two strings are anagrams. **[asked x7 in your bank]**

   Answer: Anagrams have the same length and the same characters. Sort both and compare (O(n log n)), or count frequencies in an array (O(n)).

   Example: Verify search returns the same product for "listen" and "silent" style keyword variants.

```
static boolean isAnagram(String a, String b) {
  if (a.length() != b.length()) return false;
  char[] x = a.toLowerCase().toCharArray(), y = b.toLowerCase().toCharArray();
  Arrays.sort(x); Arrays.sort(y);
  return Arrays.equals(x, y);
}
```

4. Count the occurrences (frequency) of each character in a string. **[asked x6 in your bank]**

   Answer: Use a Map<Character,Integer> and merge/increment per character; LinkedHashMap keeps the original order (O(n)).

   Example: Count how often each letter appears in a product title to check the search index.

```
static Map<Character, Integer> freq(String s) {
  Map<Character, Integer> m = new LinkedHashMap<>();
  for (char c : s.toCharArray()) m.put(c, m.getOrDefault(c, 0) + 1);
  return m;
}
```

5. Find the duplicate characters in a string. **[asked x4 in your bank]**

   Answer: Add each character to a seen set; if add() returns false it is a duplicate (O(n)).

   Example: Find repeated characters in a typed username before registration validation.

```
static Set<Character> duplicates(String s) {
  Set<Character> seen = new HashSet<>(), dup = new LinkedHashSet<>();
  for (char c : s.toCharArray()) if (!seen.add(c)) dup.add(c);
  return dup;
}
```

6. Find the first (or last) non-repeating character in a string. **[asked x4 in your bank]**

> **Answer:** Count with a LinkedHashMap (keeps order), then return the first entry with count 1. For the last non-repeating, keep the last such entry.
>
> **Example:** From a review text, find the first character that appears only once.

```
static char firstNonRepeating(String s) {
  Map<Character, Integer> m = new LinkedHashMap<>();
  for (char c : s.toCharArray()) m.merge(c, 1, Integer::sum);
  for (Map.Entry<Character, Integer> e : m.entrySet())
    if (e.getValue() == 1) return e.getKey();
  return '\0';
}
```

7. Reverse the words (or each word) in a sentence, with or without split(). **[asked x6 in your bank]**

> **Answer:** Split on whitespace and print from the last word to the first. To reverse each word instead, reverse every token and keep order.
>
> **Example:** Reverse the words of a product title such as "Blue Cotton T Shirt".

```
static String reverseWords(String s) {
  String[] w = s.trim().split("\\s+");
  StringBuilder sb = new StringBuilder();
  for (int i = w.length - 1; i >= 0; i--) sb.append(w[i]).append(i > 0 ? " " : "");
  return sb.toString();
}
```

8. Count vowels and consonants (and digits and spaces) in a string. **[asked x6 in your bank]**

> **Answer:** Lower-case the string and classify each character: vowel, other letter (consonant), digit or space (O(n)).
>
> **Example:** Analyse a product description for letters, digits and spaces in a data-quality check.

```
static void count(String s) {
  int v = 0, c = 0, d = 0, sp = 0;
  for (char ch : s.toLowerCase().toCharArray()) {
    if ("aeiou".indexOf(ch) >= 0) v++;
    else if (Character.isLetter(ch)) c++;
    else if (Character.isDigit(ch)) d++;
    else if (ch == ' ') sp++;
  }
  System.out.println(v + " " + c + " " + d + " " + sp);
}
```

9. Remove duplicate characters from a string (print unique characters). **[asked x3 in your bank]**

> **Answer:** Add characters to a LinkedHashSet, which drops duplicates and keeps the first-occurrence order (O(n)).
>
> **Example:** Clean repeated characters out of a typed tag or keyword.

```
static String unique(String s) {
  Set<Character> set = new LinkedHashSet<>();
  for (char c : s.toCharArray()) set.add(c);
  StringBuilder sb = new StringBuilder();
  for (char c : set) sb.append(c);
  return sb.toString();
}
```

10. Remove white spaces (and special characters) from a string, without built-in methods. **[asked x5 in your bank]**

> **Answer:** Loop through the characters and append only letters and digits to a StringBuilder. The built-in shortcut is replaceAll("[^a-zA-Z0-9]", "").
>
> **Example:** Sanitize a phone number field entered as "+91 98-765 43210".

```
static String clean(String s) {
  StringBuilder sb = new StringBuilder();
  for (char c : s.toCharArray())
    if (Character.isLetterOrDigit(c)) sb.append(c);
  return sb.toString();
}
```

Number & Math Programs (5)

11. Print the Fibonacci series (up to n terms or a given range). **[asked x7 in your bank]**

> **Answer:** Keep two previous values and add them each step (O(n), O(1) space). Recursion is simpler but exponential unless memoized.
>
> **Example:** Generate a sequence of 10 numbers as test data for pagination checks.

```
static void fib(int n) {
  int a = 0, b = 1;
  for (int i = 0; i < n; i++) {
    System.out.print(a + " ");
    int t = a + b; a = b; b = t;
  }
}
```

12. Find the factorial of a number (loop and recursion). **[asked x4 in your bank]**

> **Answer:** Multiply numbers from 2 to n in a loop, or use recursion with base case n <= 1. Use long (factorials overflow quickly).
>
> **Example:** Count the possible arrangements of n items in a bundle-offer combination test.

```
static long factLoop(int n) {
  long r = 1;
  for (int i = 2; i <= n; i++) r *= i;
  return r;
}
static long fact(int n) { return n <= 1 ? 1 : n * fact(n - 1); }   // recursion
```

13. Check whether a number is prime. **[asked x5 in your bank]**

> **Answer:** Check divisibility from 2 up to sqrt(n); numbers below 2 are not prime (O(sqrt n)).
>
> **Example:** Pick only prime-numbered product IDs when building sample test data.

```
static boolean isPrime(int n) {
  if (n < 2) return false;
  for (int i = 2; (long) i * i <= n; i++) if (n % i == 0) return false;
  return true;
}
```

14. Swap two numbers without a third variable. **[asked x3 in your bank]**

> **Answer:** Use addition and subtraction, or XOR. Both avoid a temporary variable (watch for overflow with addition).
>
> **Example:** Swap the old and new price values while testing a price-update comparison.

```
static void swap() {
  int a = 5, b = 9;
  a = a + b; b = a - b; a = a - b;   // or: a ^= b; b ^= a; a ^= b;
  System.out.println(a + " " + b);   // 9 5
}
```

15. Check whether a number is a palindrome. **[asked x3 in your bank]**

> **Answer:** Reverse the number using digit extraction (% 10 and / 10) and compare with the original (O(log n)).
>
> **Example:** Check whether an order number such as 12321 is a palindrome.

```
static boolean isPalNum(int n) {
  int r = 0, t = n;
  while (t > 0) { r = r * 10 + t % 10; t /= 10; }
  return r == n;
}
```

Array Programs (6)

16. Find the largest and smallest (min and max) number in an array. **[asked x8 in your bank]**

> **Answer:** Start with the first element and update min and max in a single pass (O(n)).
>
> **Example:** Find the cheapest and the most expensive price in a product list.

```
static void minMax(int[] a) {
  int min = a[0], max = a[0];
  for (int x : a) { if (x < min) min = x; if (x > max) max = x; }
  System.out.println(min + " " + max);
}
```

17. Find the second largest (second highest) number in an array. **[asked x3 in your bank]**

> **Answer:** Track first and second largest in one pass; skip values equal to the largest so duplicates do not count (O(n)).
>
> **Example:** Find the second highest price on a category page.

```
static int secondLargest(int[] a) {
  int first = Integer.MIN_VALUE, second = Integer.MIN_VALUE;
  for (int x : a) {
    if (x > first) { second = first; first = x; }
    else if (x > second && x != first) second = x;
  }
  return second;
}
```

18. Remove duplicates from an array. **[asked x2 in your bank]**

> **Answer:** Add elements to a LinkedHashSet to keep order and drop duplicates (O(n)). Without extra space, sort first and compare neighbours.
>
> **Example:** Remove duplicate product IDs from a bulk-upload file.

```
static Integer[] removeDup(int[] a) {
  Set<Integer> s = new LinkedHashSet<>();
  for (int x : a) s.add(x);
  return s.toArray(new Integer[0]);
}
```

19. Find duplicate elements in an array. **[asked x4 in your bank]**

> **Answer:** Keep a seen set; an element that cannot be added again is a duplicate (O(n)).
>
> **Example:** Find repeated SKUs in a cart or catalogue feed.

```
static Set<Integer> findDup(int[] a) {
  Set<Integer> seen = new HashSet<>(), dup = new LinkedHashSet<>();
  for (int x : a) if (!seen.add(x)) dup.add(x);
  return dup;
}
```

20. Find the missing number in an array (1 to n) or in a continuous sequence. **[asked x5 in your bank]**

> **Answer:** For numbers 1 to n with one missing, expected sum n(n+1)/2 minus the actual sum gives the missing number (O(n)).
>
> **Example:** Find the missing order number in a sequence of 1 to 100.

```
static int missing(int[] a, int n) {   // a holds n-1 numbers from 1..n
  int sum = n * (n + 1) / 2;
  for (int x : a) sum -= x;
  return sum;
}
```

21. Reverse an array.

> **Answer:** Swap the first and last elements and move both ends inward (O(n), in place).
>
> **Example:** Display the latest viewed products first by reversing the history array.

```
static void reverse(int[] a) {
  for (int i = 0, j = a.length - 1; i < j; i++, j--) {
    int t = a[i]; a[i] = a[j]; a[j] = t;
  }
}
```

Collections & HashMap Programs (incl. Test Data Handling) (5)

22. Remove duplicates from an ArrayList or List (using Set, keeping the order). **[asked x6 in your bank]**

> **Answer:** Put the list into a LinkedHashSet and back into an ArrayList; it removes duplicates and keeps order (O(n)).
>
> **Example:** De-duplicate product names collected from multiple search pages.

```
static List<String> dedupe(List<String> list) {
  return new ArrayList<>(new LinkedHashSet<>(list));
}
```

23. Iterate through a HashMap and print key-value pairs.

> **Answer:** Loop over entrySet() and print getKey() and getValue().
>
> **Example:** Print each SKU and quantity in the cart map.

```
static void print(Map<String, Integer> m) {
  for (Map.Entry<String, Integer> e : m.entrySet())
    System.out.println(e.getKey() + " = " + e.getValue());
}
```

24. Find the frequency of each element using a Map.

> **Answer:** Use merge(x, 1, Integer::sum) to count occurrences of each element in one pass (O(n)).
>
> **Example:** Count how many times each product was added to carts.

```
static Map<Integer, Integer> freq(int[] a) {
  Map<Integer, Integer> m = new HashMap<>();
  for (int x : a) m.merge(x, 1, Integer::sum);
  return m;
}
```

25. Sort a HashMap by keys and by values. **[asked x3 in your bank]**

> **Answer:** By key: copy into a TreeMap. By value: sort the entrySet with comparingByValue() (HashMap itself has no order).
>
> **Example:** Sort a SKU-to-price map by SKU and then by price to verify the sort options.

```
static void sortMap(Map<String, Integer> m) {
  new TreeMap<>(m).forEach((k, v) -> System.out.println(k + "=" + v));   // by key
  m.entrySet().stream().sorted(Map.Entry.comparingByValue())
   .forEach(e -> System.out.println(e.getKey() + "=" + e.getValue()));   // by value
}
```

26. Sort a collection of employee objects (by ID, name, department, age) using Comparable/Comparator. **[asked x2 in your bank]**

> **Answer:** Implement Comparable for a default order, or pass a Comparator for any field (id, name, age). list.sort(Comparator) handles it.
>
> **Example:** Sort customer records by ID, then by name to check an admin grid.

```
static class Employee {
  int id; String name; int age;
  Employee(int id, String name, int age) { this.id = id; this.name = name; this.age = age; }
}
static void sortEmp(List<Employee> l) {
  l.sort(Comparator.comparingInt((Employee e) -> e.id));      // by id
  l.sort(Comparator.comparing((Employee e) -> e.name));        // by name
}
```

Sorting, Searching & Complexity (1)

27. Sort an array without using built-in methods (Bubble sort). **[asked x3 in your bank]**

> **Answer:** Repeatedly swap adjacent out-of-order elements; each pass bubbles the largest value to the end (O(n^2), O(1) space).
>
> **Example:** Sort a small list of prices without using built-in sort to understand the logic.

```
static void bubble(int[] a) {
  for (int i = 0; i < a.length - 1; i++)
    for (int j = 0; j < a.length - 1 - i; j++)
      if (a[j] > a[j + 1]) { int t = a[j]; a[j] = a[j + 1]; a[j + 1] = t; }
}
```

Section 2: High Priority (63 questions)

Frequent follow-ups and the next level of difficulty. Practise these once you can do Section 1 without hesitation, with focus on HashMap, Streams and file handling.

String Programs (14)

28. Count the number of words in a string. **[asked x3 in your bank]**

> **Answer:** Trim the string, return 0 if empty, otherwise split on one or more spaces and take the length (O(n)).
>
> **Example:** Count the words in a product title to check a 60-word description limit.

```
static int wordCount(String s) {
  s = s.trim();
  return s.isEmpty() ? 0 : s.split("\\s+").length;
}
```

29. Count the occurrences of each word in a sentence (use HashMap) and find duplicate words. **[asked x2 in your bank]**

> **Answer:** Split into words and count with a map; words whose count is above 1 are the duplicates (O(n)).
>
> **Example:** Find repeated words in a customer review.

```
static Map<String, Integer> wordFreq(String s) {
  Map<String, Integer> m = new LinkedHashMap<>();
  for (String w : s.toLowerCase().split("\\s+")) m.merge(w, 1, Integer::sum);
  return m;   // entries with value > 1 are duplicate words
}
```

30. Find the longest substring without repeating characters. **[asked x3 in your bank]**

> **Answer:** Sliding window: store each character's last index; when a repeat falls inside the window, move the start past it (O(n)).
>
> **Example:** Find the longest run of unique characters in a generated order token.

```
static int longestUnique(String s) {
  Map<Character, Integer> last = new HashMap<>();
  int best = 0, start = 0;
  for (int i = 0; i < s.length(); i++) {
    char c = s.charAt(i);
    if (last.containsKey(c) && last.get(c) >= start) start = last.get(c) + 1;
    last.put(c, i);
    best = Math.max(best, i - start + 1);
  }
  return best;
}
```

31. Separate letters, digits, uppercase and lowercase characters from a mixed string (e.g. "aBACbcEDed" to "abcde" and "ABCDE"; "Subbu123raj" to "Subburaj" and "123"). **[asked x3 in your bank]**

> **Answer:** Check each character with Character.isUpperCase/isLowerCase/isDigit and append to separate StringBuilders (O(n)).
>
> **Example:** Split an alphanumeric coupon like "Save20Now" into letters and digits.

```
static void split(String s) {
  StringBuilder up = new StringBuilder(), lo = new StringBuilder();
  StringBuilder letters = new StringBuilder(), dg = new StringBuilder();
  for (char c : s.toCharArray()) {
    if (Character.isLetter(c)) letters.append(c);
    if (Character.isUpperCase(c)) up.append(c);
    else if (Character.isLowerCase(c)) lo.append(c);
    else if (Character.isDigit(c)) dg.append(c);
  }
  System.out.println(letters + " | " + up + " | " + lo + " | " + dg);
}
```

32. Compress a string (run-length encoding), e.g. "aabbcccdd" to "a2b2c3d2". **[asked x2 in your bank]**

> **Answer:** Count consecutive equal characters and append the character followed by its count (O(n)).
>
> **Example:** Compress a long repeated-character log or token before storing it.

```
static String compress(String s) {
  StringBuilder sb = new StringBuilder();
  for (int i = 0; i < s.length(); ) {
    int j = i;
    while (j < s.length() && s.charAt(j) == s.charAt(i)) j++;
    sb.append(s.charAt(i)).append(j - i);
    i = j;
  }
  return sb.toString();
}
```

33. Find all permutations of a string. **[asked x2 in your bank]**

> **Answer:** Recursion: fix one character, permute the remaining ones, and collect when nothing remains (O(n * n!)).
>
> **Example:** Generate all keyword orderings for a search-term combination test.

```
static void permute(String pre, String rest, List<String> out) {
  if (rest.isEmpty()) { out.add(pre); return; }
  for (int i = 0; i < rest.length(); i++)
    permute(pre + rest.charAt(i),
            rest.substring(0, i) + rest.substring(i + 1), out);
}
```

34. Check if one string is a rotation of another. **[asked x2 in your bank]**

> **Answer:** B is a rotation of A if both have the same length and A+A contains B (O(n)).
>
> **Example:** Check whether a rotated serial number matches the original.

```
static boolean isRotation(String a, String b) {
  return a.length() == b.length() && (a + a).contains(b);
}
```

35. Swap two strings without a third variable. **[asked x3 in your bank]**

> **Answer:** Concatenate both, then cut out each part using the original lengths with substring.
>
> **Example:** Swap the billing and shipping city strings without a temp variable.

```
static void swapStrings() {
  String a = "hello", b = "world";
  a = a + b;
  b = a.substring(0, a.length() - b.length());
  a = a.substring(b.length());
  System.out.println(a + " " + b);   // world hello
}
```

36. Find the longest palindromic substring. **[asked x3 in your bank]**

> **Answer:** Expand around every centre (odd and even) and keep the longest palindrome (O(n^2)).
>
> **Example:** Find the longest palindromic part of a generated reference ID.

```
static String longestPal(String s) {
  int st = 0, len = 0;
  for (int c = 0; c < s.length(); c++)
    for (int k = 0; k < 2; k++) {
      int l = c, r = c + k;
      while (l >= 0 && r < s.length() && s.charAt(l) == s.charAt(r)) { l--; r++; }
      if (r - l - 1 > len) { len = r - l - 1; st = l + 1; }
    }
  return s.substring(st, st + len);
}
```

37. Convert a string to an integer (atoi) and check whether a string contains only digits or is a number (regex). **[asked x5 in your bank]**

> **Answer:** Build the number digit by digit with n = n*10 + (c - '0'), handling a minus sign. To check for digits only, use matches("-?\\d+").
>
> **Example:** Validate that the quantity field contains only digits before converting it.

```
static int atoi(String s) {
  int n = 0, i = 0, sign = 1;
  if (s.charAt(0) == '-') { sign = -1; i++; }
  for (; i < s.length(); i++) n = n * 10 + (s.charAt(i) - '0');
  return sign * n;
}
static boolean isNumber(String s) { return s.matches("-?\\d+"); }
```

38. Convert lowercase to uppercase without using toUpperCase(). **[asked x2 in your bank]**

> **Answer:** In ASCII a lowercase letter is 32 higher than its uppercase; subtract 32 for characters between 'a' and 'z'.
>
> **Example:** Convert a coupon code to upper case without using the built-in method.

```
static String toUpper(String s) {
  StringBuilder sb = new StringBuilder();
  for (char c : s.toCharArray())
    sb.append(c >= 'a' && c <= 'z' ? (char) (c - 32) : c);
  return sb.toString();
}
```

39. Find the longest and the shortest word in a sentence. **[asked x2 in your bank]**

> **Answer:** Split into words and track the longest and shortest by length in one pass (O(n)).
>
> **Example:** Find the longest and shortest word in a product name.

```
static void longestShortest(String s) {
  String lo = null, sh = null;
  for (String w : s.split("\\s+")) {
    if (lo == null || w.length() > lo.length()) lo = w;
    if (sh == null || w.length() < sh.length()) sh = w;
  }
  System.out.println(lo + " " + sh);
}
```

40. Find the most frequent character in a string.

> **Answer:** Count with a map and keep track of the character with the highest count (O(n)).
>
> **Example:** Find the most repeated character in a review for a spam check.

```
static char mostFrequent(String s) {
  Map<Character, Integer> m = new HashMap<>();
  char best = s.charAt(0);
  for (char c : s.toCharArray()) {
    m.merge(c, 1, Integer::sum);
    if (m.get(c) > m.get(best)) best = c;
  }
  return best;
}
```

41. Check for balanced parentheses in a string.

> **Answer:** Use a stack: push opening brackets, and on a closing bracket pop and check that it matches (O(n)).
>
> **Example:** Validate that a filter expression such as "((brand=A)&(price<500))" is well formed.

```
static boolean balanced(String s) {
  Deque<Character> st = new ArrayDeque<>();
  for (char c : s.toCharArray()) {
    if ("([{".indexOf(c) >= 0) st.push(c);
    else if (")]}".indexOf(c) >= 0) {
      if (st.isEmpty()) return false;
      if ("([{".indexOf(st.pop()) != ")]}".indexOf(c)) return false;
    }
  }
  return st.isEmpty();
}
```

Number & Math Programs (11)

42. Find the sum of digits of a number (loop and recursion). **[asked x5 in your bank]**

> **Answer:** Add n % 10 and continue with n / 10 until n is 0. Recursion: n % 10 + sum(n / 10).
>
> **Example:** Calculate a checksum digit sum for an order number.

```
static int sumDigits(int n) {
  int sum = 0;
  while (n > 0) { sum += n % 10; n /= 10; }
  return sum;
}
static int sumRec(int n) { return n == 0 ? 0 : n % 10 + sumRec(n / 10); }
```

43. Check whether a number is an Armstrong number. **[asked x3 in your bank]**

> **Answer:** A number equals the sum of its digits each raised to the power of the digit count (153 = 1^3 + 5^3 + 3^3).
>
> **Example:** Generate special-number test data for a validation rule.

```
static boolean armstrong(int n) {
  int d = String.valueOf(n).length(), sum = 0, t = n;
  while (t > 0) { sum += (int) Math.pow(t % 10, d); t /= 10; }
  return sum == n;
}
```

44. Reverse a number.

> **Answer:** Take the last digit each time and build the reversed number as r = r * 10 + digit.
>
> **Example:** Reverse a numeric OTP to prepare an invalid-OTP test case.

```
static int reverseNum(int n) {
  int r = 0;
  while (n != 0) { r = r * 10 + n % 10; n /= 10; }
  return r;
}
```

45. Check whether a number is odd or even; count digits in a number. **[asked x2 in your bank]**

> **Answer:** Even if n % 2 == 0, otherwise odd. Count digits by dividing by 10 in a loop or using String.valueOf(n).length().
>
> **Example:** Decide whether an order ID is odd or even for a split-traffic rule.

```
static void check(int n) {
  System.out.println(n % 2 == 0 ? "even" : "odd");
  System.out.println(String.valueOf(Math.abs(n)).length());   // digit count
}
```

46. FizzBuzz: print "fizz", "buzz" or "fizzbuzz" for multiples of 3, 5 or both.

> **Answer:** Loop 1 to n: multiples of both 3 and 5 first, then 3, then 5, else the number.
>
> **Example:** Apply "Buy 3 get 1 free" and "every 5th order" style rules in test data.

```
static void fizzBuzz(int n) {
  for (int i = 1; i <= n; i++) {
    if (i % 15 == 0) System.out.println("fizzbuzz");
    else if (i % 3 == 0) System.out.println("fizz");
    else if (i % 5 == 0) System.out.println("buzz");
    else System.out.println(i);
  }
}
```

47. Generate random numbers within a given range.

> **Answer:** Use Random.nextInt(max - min + 1) + min, or ThreadLocalRandom.current().nextInt(min, max + 1).
>
> **Example:** Generate a random quantity between 1 and 10 for cart tests.

```
static int random(int min, int max) {
  return new Random().nextInt(max - min + 1) + min;
}
```

48. Convert a decimal number to binary, octal and hexadecimal. **[asked x2 in your bank]**

> **Answer:** Built-ins: Integer.toBinaryString, toOctalString, toHexString. Manually, divide by the base and collect remainders in reverse.
>
> **Example:** Convert an order number to hexadecimal for a tracking-code format.

```
static void convert(int n) {
  System.out.println(Integer.toBinaryString(n) + " " + Integer.toOctalString(n)
      + " " + Integer.toHexString(n));
}
static String toBinary(int n) {   // manual
  String r = "";
  for (; n > 0; n /= 2) r = (n % 2) + r;
  return r;
}
```

49. Find the GCD and LCM of two numbers.

> **Answer:** GCD with Euclid's algorithm (gcd(a, b) = gcd(b, a % b)); LCM = a / gcd * b.
>
> **Example:** Find the common pack size when two products are sold in different bundle quantities.

```
static int gcd(int a, int b) { return b == 0 ? a : gcd(b, a % b); }
static int lcm(int a, int b) { return a / gcd(a, b) * b; }
```

50. Check whether a number is a perfect number.

> **Answer:** A perfect number equals the sum of its proper divisors (6 = 1 + 2 + 3).
>
> **Example:** Generate sample numbers for a validation rule that accepts only special values.

```
static boolean isPerfect(int n) {
  int sum = 0;
  for (int i = 1; i <= n / 2; i++) if (n % i == 0) sum += i;
  return n > 1 && sum == n;
}
```

51. Check whether a given number is binary. **[asked x2 in your bank]**

> **Answer:** A number is binary if every digit is 0 or 1; check digits with % 10.
>
> **Example:** Validate that a toggle-flags field contains only 0 and 1.

```
static boolean isBinary(int n) {
  while (n > 0) { if (n % 10 > 1) return false; n /= 10; }
  return true;
}
```

52. Check whether a year is a leap year.

> **Answer:** Leap year: divisible by 4 and not by 100, or divisible by 400.
>
> **Example:** Validate February 29 delivery dates in the date picker.

```
static boolean isLeap(int y) {
  return (y % 4 == 0 && y % 100 != 0) || y % 400 == 0;
}
```

Array Programs (13)

53. Find the top two maximum numbers (or the sum of the two largest) in an array. **[asked x4 in your bank]**

> **Answer:** Track the two largest values in a single pass and add them for the sum (O(n)).
>
> **Example:** Find the two highest-priced items on a page.

```
static int sumTop2(int[] a) {
  int m1 = Integer.MIN_VALUE, m2 = Integer.MIN_VALUE;
  for (int x : a) {
    if (x > m1) { m2 = m1; m1 = x; }
    else if (x > m2) m2 = x;
  }
  return m1 + m2;
}
```

54. Print the even numbers from an array; count odd and even numbers. **[asked x3 in your bank]**

> **Answer:** Loop and test x % 2 == 0, printing and counting evens; odds = length - evens (O(n)).
>
> **Example:** Count even and odd product IDs in a result set.

```
static void evenOdd(int[] a) {
  int even = 0;
  for (int x : a) if (x % 2 == 0) { System.out.print(x + " "); even++; }
  System.out.println("\nEven=" + even + " Odd=" + (a.length - even));
}
```

55. Find common elements (intersection) between two arrays. **[asked x4 in your bank]**

> **Answer:** Put one array in a HashSet and keep elements of the other found in it (O(n + m)).
>
> **Example:** Find products common to two wishlists.

```
static Set<Integer> common(int[] a, int[] b) {
  Set<Integer> s = new HashSet<>(), out = new LinkedHashSet<>();
  for (int x : a) s.add(x);
  for (int x : b) if (s.contains(x)) out.add(x);
  return out;
}
```

56. Search for an element in an array. **[asked x2 in your bank]**

> **Answer:** Linear search returns the index of the first match or -1 (O(n)); use binary search if the array is sorted.
>
> **Example:** Check whether a product ID exists in the result array.

```
static int search(int[] a, int key) {
  for (int i = 0; i < a.length; i++) if (a[i] == key) return i;
  return -1;
}
```

57. Sum only the integers in an array that has integers and special characters.

> **Answer:** Treat each item as text and add only those that match an integer pattern (O(n)).
>
> **Example:** Sum the numeric values in a row that also contains symbols like "#" or "N/A".

```
static int sumIntegers(String[] a) {
  int sum = 0;
  for (String s : a) if (s.matches("-?\\d+")) sum += Integer.parseInt(s);
  return sum;
}
```

58. Find the non-repeated elements in an array, e.g. [1,1,2,2,3,4,5,5,6,6] gives [3,4]. **[asked x2 in your bank]**

> **Answer:** Count with a LinkedHashMap and return elements whose count is 1 (O(n)).
>
> **Example:** Find product IDs that appear only once in a data feed.

```
static List<Integer> nonRepeated(int[] a) {
  Map<Integer, Integer> m = new LinkedHashMap<>();
  for (int x : a) m.merge(x, 1, Integer::sum);
  List<Integer> out = new ArrayList<>();
  for (Map.Entry<Integer, Integer> e : m.entrySet())
    if (e.getValue() == 1) out.add(e.getKey());
  return out;
}
```

59. Find the first repeating element in an array. **[asked x2 in your bank]**

> **Answer:** Scan from the left with a HashSet; the first element that is already present is the first repeating one (O(n)).
>
> **Example:** Find the first duplicate item added to the cart.

```
static int firstRepeating(int[] a) {
  Set<Integer> s = new HashSet<>();
  for (int x : a) if (!s.add(x)) return x;
  return -1;
}
```

60. Move all zeroes to the end (or start) of an array, e.g. "32400121200" to "32412120000". **[asked x3 in your bank]**

> **Answer:** Copy non-zero values forward, then fill the rest with zeros (O(n), in place, order preserved).
>
> **Example:** Push zero-quantity items to the end of a cart summary.

```
static void moveZeros(int[] a) {
  int k = 0;
  for (int x : a) if (x != 0) a[k++] = x;
  while (k < a.length) a[k++] = 0;
}
```

61. Check whether two arrays are equal (irrespective of order). **[asked x2 in your bank]**

> **Answer:** Sort copies of both arrays and compare with Arrays.equals (O(n log n)).
>
> **Example:** Verify that two carts contain the same items regardless of order.

```
static boolean sameElements(int[] a, int[] b) {
  int[] x = a.clone(), y = b.clone();
  Arrays.sort(x); Arrays.sort(y);
  return Arrays.equals(x, y);
}
```

62. Find all pairs of elements whose sum equals a given number.

> **Answer:** For each element look for (target - element) in a set of values already seen (O(n)).
>
> **Example:** Find two items whose prices add up to a gift-card value.

```
static void pairs(int[] a, int target) {
  Set<Integer> seen = new HashSet<>();
  for (int x : a) {
    if (seen.contains(target - x)) System.out.println(x + "," + (target - x));
    seen.add(x);
  }
}
```

63. Rotate an array by a given number of steps.

> **Answer:** Right-rotate by k using three reversals: whole array, first k, then the rest (O(n), O(1) space).
>
> **Example:** Rotate a banner/carousel list of products by k positions.

```
static void rotate(int[] a, int k) {
  k %= a.length;
  rev(a, 0, a.length - 1); rev(a, 0, k - 1); rev(a, k, a.length - 1);
}
static void rev(int[] a, int i, int j) {
  while (i < j) { int t = a[i]; a[i++] = a[j]; a[j--] = t; }
}
```

64. Merge two sorted arrays. **[asked x2 in your bank]**

> **Answer:** Use two pointers, always copying the smaller current element, then copy what is left (O(n + m)).
>
> **Example:** Merge two price-sorted lists from different sellers.

```
static int[] merge(int[] a, int[] b) {
  int[] r = new int[a.length + b.length];
  int i = 0, j = 0, k = 0;
  while (i < a.length && j < b.length) r[k++] = a[i] <= b[j] ? a[i++] : b[j++];
  while (i < a.length) r[k++] = a[i++];
  while (j < b.length) r[k++] = b[j++];
  return r;
}
```

65. Remove a given element from an array; copy an array. **[asked x2 in your bank]**

> **Answer:** Arrays have a fixed size, so filter into a new array. Copy with Arrays.copyOf or clone().
>
> **Example:** Remove a deleted product ID from an ID array.

```
static int[] remove(int[] a, int val) {
  return Arrays.stream(a).filter(x -> x != val).toArray();
}
static int[] copy(int[] a) { return Arrays.copyOf(a, a.length); }
```

Collections & HashMap Programs (incl. Test Data Handling) (8)

66. How do you store and retrieve test data dynamically using a Map? How do you handle a key with multiple values? **[asked x2 in your bank]**

> **Answer:** Use Map<String, String> for single values and Map<String, List<String>> with computeIfAbsent when a key has multiple values.
>
> **Example:** Store checkout test data by key and keep several card numbers under "cards".

```
static void testData() {
  Map<String, List<String>> data = new HashMap<>();
  data.computeIfAbsent("browser", k -> new ArrayList<>()).add("chrome");
  data.computeIfAbsent("browser", k -> new ArrayList<>()).add("firefox");
  System.out.println(data.get("browser"));   // [chrome, firefox]
}
```

67. Convert a HashMap to a List.

> **Answer:** Wrap the entrySet, keySet or values in a new ArrayList.
>
> **Example:** Convert the cart map into a list to loop over by index.

```
static void toList(Map<String, Integer> m) {
  List<Map.Entry<String, Integer>> entries = new ArrayList<>(m.entrySet());
  List<String> keys = new ArrayList<>(m.keySet());
  List<Integer> values = new ArrayList<>(m.values());
}
```

68. Sort a list alphabetically, or in reverse order. **[asked x2 in your bank]**

> **Answer:** Collections.sort(list) for ascending; pass Collections.reverseOrder() or Comparator.reverseOrder() for descending.
>
> **Example:** Verify product names are shown A-Z and Z-A.

```
static void sortList(List<String> l) {
  Collections.sort(l);                          // A-Z
  l.sort(Comparator.reverseOrder());            // Z-A
}
```

69. Find the first and last element of an ArrayList.

> **Answer:** Use get(0) for the first and get(size() - 1) for the last element (Java 21 adds getFirst/getLast).
>
> **Example:** Check the first and last product on a results page.

```
static void ends(List<String> l) {
  String first = l.get(0), last = l.get(l.size() - 1);
  System.out.println(first + " " + last);
}
```

70. Merge two lists and remove duplicates.

> **Answer:** Add both lists to a LinkedHashSet to merge them without duplicates and keep order.
>
> **Example:** Combine recommended and recently-viewed products without repeats.

```
static List<String> mergeLists(List<String> a, List<String> b) {
  Set<String> s = new LinkedHashSet<>(a);
  s.addAll(b);
  return new ArrayList<>(s);
}
```

71. Identify the first non-repeated element using LinkedHashMap.

> **Answer:** Count into a LinkedHashMap (insertion order), then return the first key with count 1 (O(n)).
>
> **Example:** Find the first product ID that was clicked only once.

```
static Integer firstUnique(List<Integer> l) {
  Map<Integer, Integer> m = new LinkedHashMap<>();
  for (int x : l) m.merge(x, 1, Integer::sum);
  for (Map.Entry<Integer, Integer> e : m.entrySet())
    if (e.getValue() == 1) return e.getKey();
  return null;
}
```

72. Remove null or empty values from a collection.

> **Answer:** Use removeIf with a null/empty check.
>
> **Example:** Clean blank values from a coupon-code list read from a file.

```
static void clean(List<String> l) {
  l.removeIf(s -> s == null || s.trim().isEmpty());
}
```

73. Iterate over a list without using loops.

> **Answer:** Use forEach with a lambda or method reference (a stream also works); no explicit loop is written.
>
> **Example:** Print all product names without a for loop.

```
static void print(List<String> l) {
  l.forEach(System.out::println);
}
```

Java 8 Streams Programs (5)

74. Use Java 8 Streams to filter data (e.g. even numbers, test data) and count elements that match a condition. **[asked x3 in your bank]**

> **Answer:** Use stream().filter(condition) and then collect or count().
>
> **Example:** Count in-stock products or collect even-numbered test IDs.

```
static void filter(List<Integer> nums) {
  List<Integer> even = nums.stream().filter(n -> n % 2 == 0).collect(Collectors.toList());
  long count = nums.stream().filter(n -> n > 100).count();
  System.out.println(even + " " + count);
}
```

75. Transform a list of names to uppercase using Streams.

> **Answer:** Use stream().map(String::toUpperCase) and collect to a list.
>
> **Example:** Upper-case category names before comparing with the UI text.

```
static List<String> upper(List<String> names) {
  return names.stream().map(String::toUpperCase).collect(Collectors.toList());
}
```

76. Find the frequency of elements using Streams.

> **Answer:** Group by the element itself and count: groupingBy(Function.identity(), counting()).
>
> **Example:** Count how many times each product appears in the orders list.

```
static Map<String, Long> freq(List<String> l) {
  return l.stream().collect(Collectors.groupingBy(Function.identity(), Collectors.counting()));
}
```

77. Collect stream results into a List or Set, and convert a list to a map. **[asked x2 in your bank]**

> **Answer:** collect(Collectors.toList()/toSet()) for collections; toMap(keyFn, valueFn) for a map (keys must be unique).
>
> **Example:** Build a SKU-to-name map from the product list.

```
static void collect(List<String> l) {
  List<String> list = l.stream().collect(Collectors.toList());
  Set<String> set = l.stream().collect(Collectors.toSet());
  Map<String, Integer> lengths = l.stream().distinct()
      .collect(Collectors.toMap(s -> s, String::length));
}
```

78. Remove duplicates from a list using Streams.

> **Answer:** Use distinct() in the stream pipeline.
>
> **Example:** Remove duplicate brand names from the filter panel list.

```
static List<String> dedupe(List<String> l) {
  return l.stream().distinct().collect(Collectors.toList());
}
```

Sorting, Searching & Complexity (2)

79. Implement binary search.

> **Answer:** On a sorted array, compare with the middle element and discard half each time (O(log n)).
>
> **Example:** Find a price in a price-sorted list quickly.

```
static int binarySearch(int[] a, int key) {
  int lo = 0, hi = a.length - 1;
  while (lo <= hi) {
    int mid = (lo + hi) >>> 1;
    if (a[mid] == key) return mid;
    if (a[mid] < key) lo = mid + 1; else hi = mid - 1;
  }
  return -1;
}
```

80. Implement Selection sort.

> **Answer:** Find the minimum of the unsorted part and swap it into the next position (O(n^2), few swaps).
>
> **Example:** Sort a short list of delivery slots by time.

```
static void selection(int[] a) {
  for (int i = 0; i < a.length - 1; i++) {
    int min = i;
    for (int j = i + 1; j < a.length; j++) if (a[j] < a[min]) min = j;
    int t = a[i]; a[i] = a[min]; a[min] = t;
  }
}
```

OOP, Design Pattern & Exception Coding (7)

81. Write a Singleton class / implement the Singleton design pattern. **[asked x2 in your bank]**

> **Answer:** A private constructor plus a static getInstance() ensures one instance. Double-checked locking with volatile makes lazy creation thread safe.
>
> **Example:** One ConfigReader or report manager shared across all tests.

```
class Singleton {
  private static volatile Singleton instance;
  private Singleton() { }
  static Singleton getInstance() {
    if (instance == null)
      synchronized (Singleton.class) {
        if (instance == null) instance = new Singleton();
      }
    return instance;
  }
}
```

82. Implement hashCode() and equals().

> **Answer:** Override equals() to compare the identifying fields and hashCode() using the same fields, so equal objects have equal hash codes.
>
> **Example:** Two Product objects with the same SKU are one item in a HashSet.

```
static class Product {
  String sku;
  @Override public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof Product)) return false;
    return Objects.equals(sku, ((Product) o).sku);
  }
  @Override public int hashCode() { return Objects.hash(sku); }
}
```

83. Demonstrate method overloading and overriding.

> **Answer:** Overloading: same name, different parameters (compile time). Overriding: child redefines a parent method (runtime).
>
> **Example:** addToCart(sku) and addToCart(sku, qty); UpiPayment overriding pay().

```
class Payment { void pay() { System.out.println("Pay"); } }
class Upi extends Payment {
  @Override void pay() { System.out.println("Pay by UPI"); }   // overriding
  void pay(int amount) { System.out.println("Pay " + amount); } // overloading
}
```

84. Create a simple class with a constructor and getters/setters.

> **Answer:** A class with private fields, a constructor and public getters/setters (encapsulation).
>
> **Example:** A Product class holding name and price.

```
static class Product {
  private String name; private double price;
  Product(String name, double price) { this.name = name; this.price = price; }
  String getName() { return name; }
  double getPrice() { return price; }
  void setPrice(double price) { this.price = price; }
}
```

85. Demonstrate interface and abstract class usage.

> **Answer:** An interface declares what must be done; an abstract class shares common code and leaves some methods abstract.
>
> **Example:** PaymentMethod interface and an abstract BasePage class.

```
interface PaymentMethod { void pay(double amount); }
abstract class BasePage {
  void waitForLoad() { System.out.println("loaded"); }   // shared code
  abstract String title();                                // must implement
}
```

86. Create a custom exception class.

> **Answer:** Extend Exception (checked) or RuntimeException (unchecked) and pass the message to super.
>
> **Example:** Throw OutOfStockException when a product has no stock.

```
static class OutOfStockException extends RuntimeException {
  OutOfStockException(String msg) { super(msg); }
}
static void addToCart(int stock) {
  if (stock == 0) throw new OutOfStockException("Item unavailable");
}
```

87. Can we write a method that returns two or more values? How?

> **Answer:** A Java method returns one value, so wrap several values in an array, List, Map or a small class (or a record in Java 16+).
>
> **Example:** Return both the minimum and maximum price from one method.

```
static class Range {
  final int min, max;
  Range(int min, int max) { this.min = min; this.max = max; }
}
static Range range(int[] a) {
  return new Range(Arrays.stream(a).min().getAsInt(), Arrays.stream(a).max().getAsInt());
}
```

File Handling Programs (1)

88. Read a file and print its content; read a text file and count words. **[asked x2 in your bank]**

> **Answer:** Read all lines with Files.readAllLines (or BufferedReader), print them and count words by splitting each non-empty line.
>
> **Example:** Read a text file of coupon codes and count the entries.

```
static void readFile(String path) throws IOException {
  int words = 0;
  for (String line : Files.readAllLines(Paths.get(path))) {
    System.out.println(line);
    if (!line.trim().isEmpty()) words += line.trim().split("\\s+").length;
  }
  System.out.println("Words: " + words);
}
```

Multithreading Programs (2)

89. Write a multithreading program that prints odd numbers with one thread and even numbers with another.

> **Answer:** Share one lock object; each thread prints only when it is its turn, otherwise wait(); after printing, notifyAll().
>
> **Example:** Two threads alternately process odd and even order numbers.

```
class OddEven {
  private int n = 1;
  private final int max = 10;
  synchronized void print(boolean odd) throws InterruptedException {
    while (n <= max) {
      if ((n % 2 == 1) == odd) {
        System.out.println(Thread.currentThread().getName() + ": " + n++);
        notifyAll();
      } else wait();
    }
    notifyAll();
  }
}
// new Thread(() -> { try { oe.print(true); } catch (InterruptedException e) { } }).start();
```

90. Write a program to create a deadlock between two threads and explain the deadlock condition. **[asked x2 in your bank]**

> **Answer:** Two threads take two locks in opposite order and wait forever. Conditions: mutual exclusion, hold and wait, no preemption, circular wait.
>
> **Example:** Thread 1 locks the cart then payment, thread 2 locks payment then cart.

```
static final Object CART = new Object(), PAY = new Object();
static void deadlock() {
  new Thread(() -> { synchronized (CART) { sleep(); synchronized (PAY) { } } }).start();
  new Thread(() -> { synchronized (PAY) { sleep(); synchronized (CART) { } } }).start();
}
static void sleep() { try { Thread.sleep(100); } catch (InterruptedException e) { } }
```

Section 3: Medium Priority (45 questions)

Occasionally asked, usually for SDET-leaning roles or as a challenge question. Know the approach even if you cannot write it perfectly.

String Programs (8)

91. Find the length of a string without using length().

> **Answer:** Loop over toCharArray() and count each character instead of calling length().
>
> **Example:** Check a field length limit without relying on length().

```
static int length(String s) {
  int n = 0;
  for (char c : s.toCharArray()) n++;
  return n;
}
```

92. Reverse a string while preserving the position of spaces.

> **Answer:** Reverse only the non-space characters and keep every space at its original index (O(n)).
>
> **Example:** Reverse a product title while leaving the word gaps in place.

```
static String reverseKeepSpaces(String s) {
  char[] out = s.toCharArray();
  int j = s.length() - 1;
  for (int i = 0; i < out.length; i++) {
    if (out[i] == ' ') continue;
    while (s.charAt(j) == ' ') j--;
    out[i] = s.charAt(j--);
  }
  return new String(out);
}
```

93. Find all substrings of a given string.

> **Answer:** Use two loops: start index and end index, and take substring(i, j) (O(n^2) substrings).
>
> **Example:** Generate all search-keyword fragments from a product name.

```
static void substrings(String s) {
  for (int i = 0; i < s.length(); i++)
    for (int j = i + 1; j <= s.length(); j++)
      System.out.println(s.substring(i, j));
}
```

94. Check if a string is a valid shuffle of two strings.

> **Answer:** A valid shuffle keeps the order of characters of both strings. Use a DP table where dp[i][j] means first i of A and first j of B form the first i+j of the result.
>
> **Example:** Check that merged cart items keep the relative order of both original carts.

```
static boolean isShuffle(String a, String b, String s) {
  if (a.length() + b.length() != s.length()) return false;
  boolean[][] dp = new boolean[a.length() + 1][b.length() + 1];
  dp[0][0] = true;
  for (int i = 0; i <= a.length(); i++)
    for (int j = 0; j <= b.length(); j++) {
      char c = i + j > 0 ? s.charAt(i + j - 1) : 0;
      if (i > 0 && dp[i - 1][j] && a.charAt(i - 1) == c) dp[i][j] = true;
      if (j > 0 && dp[i][j - 1] && b.charAt(j - 1) == c) dp[i][j] = true;
    }
  return dp[a.length()][b.length()];
}
```

95. Find the longest common prefix.

> **Answer:** Take the first word as the prefix and shorten it until every word starts with it (O(total characters)).
>
> **Example:** Find the common prefix of product SKUs such as "SKU-10", "SKU-11".

```
static String lcp(String[] w) {
  String p = w[0];
  for (String s : w) while (!s.startsWith(p)) p = p.substring(0, p.length() - 1);
  return p;
}
```

96. Remove a given character from a string.

> **Answer:** Build a new string skipping the given character, or use replace(String.valueOf(c), "").
>
> **Example:** Remove hyphens from a phone number.

```
static String remove(String s, char ch) {
  StringBuilder sb = new StringBuilder();
  for (char c : s.toCharArray()) if (c != ch) sb.append(c);
  return sb.toString();
}
```

97. Print even-indexed characters, or print each letter twice. **[asked x2 in your bank]**

> **Answer:** Loop and use the index: print when i % 2 == 0, or append each character twice.
>
> **Example:** Mask every other character of a card number display.

```
static void patterns(String s) {
  for (int i = 0; i < s.length(); i += 2) System.out.print(s.charAt(i));   // even index
  System.out.println();
  for (char c : s.toCharArray()) System.out.print("" + c + c);             // each twice
}
```

98. Transform 'My name is so and so' into '{My-name-is-so-and-so}'.

> **Answer:** Replace spaces with hyphens and wrap the result in braces.
>
> **Example:** Format a product title into a URL-style slug.

```
static String format(String s) {
  return "{" + String.join("-", s.split(" ")) + "}";   // {My-name-is-so-and-so}
}
```

Number & Math Programs (4)

99. Find the prime factors of a number.

> **Answer:** Divide by each factor d starting at 2 while divisible; any remainder above 1 is also a prime factor (O(sqrt n)).
>
> **Example:** Break a bundle quantity into prime factors for a packaging test.

```
static List<Integer> primeFactors(int n) {
  List<Integer> f = new ArrayList<>();
  for (int d = 2; (long) d * d <= n; d++)
    while (n % d == 0) { f.add(d); n /= d; }
  if (n > 1) f.add(n);
  return f;
}
```

100. Find the sum of the first 1000 prime numbers.

> **Answer:** Test numbers one by one with an isPrime check until 1000 primes are found and add them.
>
> **Example:** Generate prime-based IDs for sample test data.

```
static long sumFirst1000Primes() {
  long sum = 0; int count = 0;
  for (int n = 2; count < 1000; n++) {
    boolean prime = true;
    for (int i = 2; i * i <= n; i++) if (n % i == 0) { prime = false; break; }
    if (prime) { sum += n; count++; }
  }
  return sum;
}
```

101. Calculate the difference between two dates in Java.

Answer: Use java.time: ChronoUnit.DAYS.between(d1, d2) for days, or Period.between for years/months/days.

Example: Calculate the number of days between order date and delivery date.

```
static long daysBetween(LocalDate d1, LocalDate d2) {
  return ChronoUnit.DAYS.between(d1, d2);
}
```

102. Print patterns and Floyd's Triangle. **[asked x2 in your bank]**

Answer: Nested loops: the outer loop controls rows and the inner loop prints the columns. Floyd's Triangle prints consecutive numbers.

Example: Print a simple test pattern for console output validation.

```
static void floyd(int rows) {
  int n = 1;
  for (int i = 1; i <= rows; i++) {
    for (int j = 1; j <= i; j++) System.out.print(n++ + " ");
    System.out.println();
  }
}
```

Array Programs (7)

103. Find all the leaders in an integer array.

Answer: A leader is greater than everything to its right. Scan from the right keeping the maximum (O(n)).

Example: Find the days on which sales were higher than all later days.

```
static List<Integer> leaders(int[] a) {
  LinkedList<Integer> r = new LinkedList<>();
  int max = Integer.MIN_VALUE;
  for (int i = a.length - 1; i >= 0; i--)
    if (a[i] > max) { max = a[i]; r.addFirst(a[i]); }
  return r;
}
```

104. Find the middle index where the sums of both ends are equal.

Answer: Equilibrium index: the sum on the left equals the sum on the right. Keep a running left sum and compare with total - left - current (O(n)).

Example: Find the point where the revenue before and after a day is balanced.

```
static int equilibrium(int[] a) {
  int total = 0, left = 0;
  for (int x : a) total += x;
  for (int i = 0; i < a.length; i++) {
    if (left == total - left - a[i]) return i;
    left += a[i];
  }
  return -1;
}
```

105. Rearrange an array into alternating positive and negative numbers.

Answer: Split into positives and negatives, then place them alternately and append the leftovers (O(n)).

Example: Alternate positive (credit) and negative (refund) transactions.

```
static int[] alternate(int[] a) {
  List<Integer> pos = new ArrayList<>(), neg = new ArrayList<>();
  for (int x : a) (x >= 0 ? pos : neg).add(x);
  int[] r = new int[a.length];
  int p = 0, n = 0, k = 0;
  while (p < pos.size() && n < neg.size()) { r[k++] = pos.get(p++); r[k++] = neg.get(n++); }
  while (p < pos.size()) r[k++] = pos.get(p++);
  while (n < neg.size()) r[k++] = neg.get(n++);
  return r;
}
```

106. Find whether a sub-array with sum equal to zero exists.

> **Answer:** Keep prefix sums in a set; a repeated prefix sum (or a sum of 0) means a zero-sum sub-array exists (O(n)).
>
> **Example:** Detect when credits and debits cancel out within a transaction range.

```
static boolean zeroSum(int[] a) {
  Set<Integer> seen = new HashSet<>();
  int sum = 0;
  seen.add(0);
  for (int x : a) { sum += x; if (!seen.add(sum)) return true; }
  return false;
}
```

107. Find the sub-array with the maximum sum (maximal subarray). **[asked x3 in your bank]**

> **Answer:** Kadane's algorithm: at each element decide whether to extend the current sub-array or start a new one (O(n)).
>
> **Example:** Find the best consecutive-day sales window.

```
static int maxSubArray(int[] a) {
  int cur = a[0], best = a[0];
  for (int i = 1; i < a.length; i++) {
    cur = Math.max(a[i], cur + a[i]);
    best = Math.max(best, cur);
  }
  return best;
}
```

108. Find the sub-array with the largest product.

> **Answer:** Track both the running maximum and minimum product (a negative can flip them) (O(n)).
>
> **Example:** Find the best multiplier window in a discount-stacking test.

```
static int maxProduct(int[] a) {
  int max = a[0], min = a[0], best = a[0];
  for (int i = 1; i < a.length; i++) {
    int x = a[i], hi = Math.max(x, Math.max(max * x, min * x));
    min = Math.min(x, Math.min(max * x, min * x));
    max = hi;
    best = Math.max(best, max);
  }
  return best;
}
```

109. Find the length of the longest consecutive sequence.

> **Answer:** Put all numbers in a set and start counting only from numbers that have no predecessor (O(n)).
>
> **Example:** Find the longest run of consecutive order numbers.

```
static int longestConsecutive(int[] a) {
  Set<Integer> s = new HashSet<>();
  for (int x : a) s.add(x);
  int best = 0;
  for (int x : s) {
    if (s.contains(x - 1)) continue;
    int len = 1;
    while (s.contains(x + len)) len++;
    best = Math.max(best, len);
  }
  return best;
}
```

Collections & HashMap Programs (incl. Test Data Handling) (4)

110. How do you handle large data lists efficiently?

> **Answer:** Pick the right collection, avoid repeated copying, process in batches or streams, pre-size lists and read data lazily from files or DB.
>
> **Example:** Process a 1 lakh product list in batches of 1000 instead of loading all details at once.

```
static void batches(List<String> list) {
  for (int i = 0; i < list.size(); i += 1000) {
    List<String> batch = list.subList(i, Math.min(i + 1000, list.size()));
    System.out.println("Processing " + batch.size());
  }
}
```

111. Reverse traversal of a list using a Stack.

> **Answer:** Push every element onto a stack, then pop to read in reverse (LIFO).
>
> **Example:** Show the recently viewed products from the latest to the oldest.

```
static void reversePrint(List<String> l) {
  Deque<String> st = new ArrayDeque<>();
  for (String s : l) st.push(s);
  while (!st.isEmpty()) System.out.println(st.pop());
}
```

112. Write your own ArrayList implementation.

> **Answer:** Keep an Object[] and a size; grow the array (usually double) when full; check index bounds in get().
>
> **Example:** A custom list class to explain how ArrayList grows internally.

```
static class MyList<T> {
  private Object[] data = new Object[4];
  private int size;
  void add(T x) {
    if (size == data.length) data = Arrays.copyOf(data, size * 2);
    data[size++] = x;
  }
  @SuppressWarnings("unchecked")
  T get(int i) {
    if (i < 0 || i >= size) throw new IndexOutOfBoundsException();
    return (T) data[i];
  }
  int size() { return size; }
}
```

113. How do you read environment variables dynamically?

> **Answer:** Use System.getenv("NAME") for one variable or System.getenv() for the whole map; System.getProperty reads JVM properties.
>
> **Example:** Read the test environment URL or credentials from CI environment variables.

```
static String env(String name, String fallback) {
  String v = System.getenv(name);
  return v != null ? v : fallback;
}
```

Java 8 Streams Programs (3)

114. Sort a list in descending order using Streams.

> **Answer:** sorted(Comparator.reverseOrder()) then collect.
>
> **Example:** Show the highest prices first and verify against the UI.

```
static List<Integer> desc(List<Integer> l) {
  return l.stream().sorted(Comparator.reverseOrder()).collect(Collectors.toList());
}
```

115. Find the maximum value in a collection using Streams.

> **Answer:** Use stream().max(...) or mapToInt(...).max().
>
> **Example:** Find the highest price in the list.

```
static int max(List<Integer> l) {
  return l.stream().mapToInt(Integer::intValue).max().getAsInt();
}
```

116. Merge two lists using Streams.

> **Answer:** Stream.concat(a.stream(), b.stream()) then collect.
>
> **Example:** Combine two seller catalogues into one list.

```
static List<String> merge(List<String> a, List<String> b) {
  return Stream.concat(a.stream(), b.stream()).collect(Collectors.toList());
}
```

Sorting, Searching & Complexity (2)

117. How do you calculate the complexity of an algorithm?

> **Answer:** Count the basic operations as a function of input size n and keep the dominant term. One loop is O(n), nested loops O(n^2), halving O(log n), sorting O(n log n). Space complexity counts extra memory.
>
> **Example:** Explain why linear search over 1 lakh products is slower than a hash lookup.

```
for (int i = 0; i < n; i++) { }                    // O(n)
for (int i = 0; i < n; i++) for (int j = 0; j < n; j++) { }   // O(n^2)
for (int i = 1; i < n; i *= 2) { }                 // O(log n)
```

118. Sort an array in place using QuickSort (recursive and iterative). **[asked x2 in your bank]**

> **Answer:** Pick a pivot, partition smaller values left and larger right, and repeat on both parts (average O(n log n)). The iterative version uses an explicit stack of ranges.
>
> **Example:** Sort a large price list in place.

```
static int partition(int[] a, int lo, int hi) {
  int p = a[hi], i = lo;
  for (int j = lo; j < hi; j++)
    if (a[j] < p) { int t = a[i]; a[i++] = a[j]; a[j] = t; }
  int t = a[i]; a[i] = a[hi]; a[hi] = t;
  return i;
}
static void quick(int[] a, int lo, int hi) {   // recursive
  if (lo < hi) { int p = partition(a, lo, hi); quick(a, lo, p - 1); quick(a, p + 1, hi); }
}
static void quickIter(int[] a) {               // iterative
  Deque<int[]> st = new ArrayDeque<>();
  st.push(new int[]{0, a.length - 1});
  while (!st.isEmpty()) {
    int[] r = st.pop();
    if (r[0] >= r[1]) continue;
    int p = partition(a, r[0], r[1]);
    st.push(new int[]{r[0], p - 1}); st.push(new int[]{p + 1, r[1]});
  }
}
```

OOP, Design Pattern & Exception Coding (1)

119. Design a Vending Machine.

> **Answer:** Model items with price and stock, track inserted balance, and on selection check availability and funds, reduce stock and return change.
>
> **Example:** Same logic as a cart: check stock, check payment, update inventory, return change/refund.

```
static class VendingMachine {
  private final Map<String, Integer> price = new HashMap<>(), stock = new HashMap<>();
  private int balance;
  void load(String item, int p, int qty) { price.put(item, p); stock.put(item, qty); }
  void insert(int coins) { balance += coins; }
  String select(String item) {
    if (stock.getOrDefault(item, 0) == 0) return "Sold out";
    if (balance < price.get(item)) return "Insert " + (price.get(item) - balance) + " more";
    stock.merge(item, -1, Integer::sum);
    int change = balance - price.get(item); balance = 0;
    return "Dispensed " + item + ", change " + change;
  }
}
```

File Handling Programs (4)

120. Count characters, words and lines in a text file.

> **Answer:** Read all lines; lines = list size, words = split on whitespace, characters = line lengths.
>
> **Example:** Count lines, words and characters in a test-data file.

```
static void stats(String path) throws IOException {
  int lines = 0, words = 0, chars = 0;
  for (String l : Files.readAllLines(Paths.get(path))) {
    lines++; chars += l.length();
    if (!l.trim().isEmpty()) words += l.trim().split("\\s+").length;
  }
  System.out.println(lines + " " + words + " " + chars);
}
```

121. Find the word with the highest frequency in a file.

> **Answer:** Split the file into words, count in a HashMap and take the entry with the highest value.
>
> **Example:** Find the most common search keyword in a search log file.

```
static String topWord(String path) throws IOException {
  Map<String, Integer> m = new HashMap<>();
  for (String l : Files.readAllLines(Paths.get(path)))
    for (String w : l.toLowerCase().split("\\s+"))
      if (!w.isEmpty()) m.merge(w, 1, Integer::sum);
  return Collections.max(m.entrySet(), Map.Entry.comparingByValue()).getKey();
}
```

122. Append text to a file.

> **Answer:** Open the file in append mode (FileWriter with true, or Files.write with APPEND).
>
> **Example:** Append a failed order ID to a results file after each test.

```
static void append(String path, String text) throws IOException {
  Files.write(Paths.get(path), (text + System.lineSeparator()).getBytes(),
      StandardOpenOption.CREATE, StandardOpenOption.APPEND);
}
```

123. Sort a text file.

> **Answer:** Read all lines into a list, sort it and write it back.
>
> **Example:** Sort a file of SKU codes before using it as test data.

```
static void sortFile(String in, String out) throws IOException {
  List<String> lines = Files.readAllLines(Paths.get(in));
  Collections.sort(lines);
  Files.write(Paths.get(out), lines);
}
```

Multithreading Programs (2)

124. Write code to avoid deadlock when N threads access N shared resources.

> **Answer:** Always acquire locks in one fixed global order (or use tryLock with a timeout) so a circular wait cannot form.
>
> **Example:** Always lock the cart before payment in every thread.

```
static void useBoth(Object x, Object y) {
  Object first = System.identityHashCode(x) < System.identityHashCode(y) ? x : y;
  Object second = first == x ? y : x;
  synchronized (first) { synchronized (second) { /* work with both */ } }
}
```

125. Implement the Producer-Consumer pattern using wait, notify and notifyAll.

> **Answer:** A shared bounded buffer: the producer waits while it is full, the consumer waits while it is empty; both notifyAll() after changing it.
>
> **Example:** A thread adds orders to a queue while another thread processes them.

```
static class Buffer {
  private final Queue<Integer> q = new LinkedList<>();
  private final int cap = 5;
  synchronized void produce(int v) throws InterruptedException {
    while (q.size() == cap) wait();
    q.add(v); notifyAll();
  }
  synchronized int consume() throws InterruptedException {
    while (q.isEmpty()) wait();
    int v = q.poll(); notifyAll(); return v;
  }
}
```

Linked List, Stack, Matrix & Advanced Problems (8)

126. Find the middle node of a linked list in one pass.

> **Answer:** Fast and slow pointers: when the fast pointer reaches the end the slow pointer is at the middle (O(n), one pass).
>
> **Example:** Find the middle product in a linked chain of recently viewed items.

```
static class Node { int val; Node next; Node(int v) { val = v; } }
static Node middle(Node head) {
  Node slow = head, fast = head;
  while (fast != null && fast.next != null) { slow = slow.next; fast = fast.next.next; }
  return slow;
}
```

127. Detect whether a linked list contains a cycle.

> **Answer:** Floyd's cycle detection: if fast and slow pointers ever meet, there is a cycle (O(n), O(1) space).
>
> **Example:** Detect an infinite redirect loop between pages.

```
static boolean hasCycle(Node head) {   // Node as in Q126
  Node slow = head, fast = head;
  while (fast != null && fast.next != null) {
    slow = slow.next; fast = fast.next.next;
    if (slow == fast) return true;
  }
  return false;
}
```

128. Find the nth element from the end of a linked list.

> **Answer:** Move the first pointer n steps ahead, then move both until the first reaches the end; the second is the answer.
>
> **Example:** Get the 3rd-last recently viewed product.

```
static Node nthFromEnd(Node head, int n) {   // Node as in Q126
  Node first = head, second = head;
  for (int i = 0; i < n; i++) first = first.next;
  while (first != null) { first = first.next; second = second.next; }
  return second;
}
```

129. Check whether a linked list is a palindrome.

> **Answer:** Find the middle, reverse the second half and compare it with the first half (O(n), O(1) space).
>
> **Example:** Check if a sequence of codes reads the same forwards and backwards.

```
static boolean isPalindrome(Node head) {   // Node as in Q126
  Node slow = head, fast = head;
  while (fast != null && fast.next != null) { slow = slow.next; fast = fast.next.next; }
  Node prev = null;
  while (slow != null) { Node nx = slow.next; slow.next = prev; prev = slow; slow = nx; }
  for (Node a = head, b = prev; b != null; a = a.next, b = b.next)
    if (a.val != b.val) return false;
  return true;
}
```

130. Add two numbers represented by linked lists.

> **Answer:** Add digit by digit with a carry, creating a new node for each digit (digits stored in reverse order) (O(n)).
>
> **Example:** Add two large amounts stored digit by digit.

```
static Node add(Node a, Node b) {   // Node as in Q126
  Node dummy = new Node(0), cur = dummy;
  int carry = 0;
  while (a != null || b != null || carry > 0) {
    int sum = carry + (a != null ? a.val : 0) + (b != null ? b.val : 0);
    carry = sum / 10;
    cur.next = new Node(sum % 10); cur = cur.next;
    if (a != null) a = a.next;
    if (b != null) b = b.next;
  }
  return dummy.next;
}
```

131. Sort a linked list.

> **Answer:** Merge sort: split at the middle, sort both halves recursively and merge them (O(n log n)).
>
> **Example:** Sort a linked list of orders by order value.

```
static Node sort(Node h) {   // Node as in Q126
  if (h == null || h.next == null) return h;
  Node slow = h, fast = h.next;
  while (fast != null && fast.next != null) { slow = slow.next; fast = fast.next.next; }
  Node right = sort(slow.next);
  slow.next = null;
  return merge(sort(h), right);
}
static Node merge(Node a, Node b) {
  Node d = new Node(0), t = d;
  while (a != null && b != null) {
    if (a.val <= b.val) { t.next = a; a = a.next; } else { t.next = b; b = b.next; }
    t = t.next;
  }
  t.next = a != null ? a : b;
  return d.next;
}
```

132. Sort a Stack using a temporary Stack.

> **Answer:** Pop each element and move larger elements from the temp stack back before pushing it, so the temp stack stays sorted.
>
> **Example:** Sort a stack of recently viewed prices.

```
static Deque<Integer> sortStack(Deque<Integer> in) {
  Deque<Integer> tmp = new ArrayDeque<>();
  while (!in.isEmpty()) {
    int x = in.pop();
    while (!tmp.isEmpty() && tmp.peek() > x) in.push(tmp.pop());
    tmp.push(x);
  }
  return tmp;   // top is the largest
}
```

133. Transpose a matrix, add two matrices and multiply matrices. **[asked x3 in your bank]**

> **Answer:** Transpose: swap rows and columns. Add: add matching cells. Multiply: result[i][j] is the sum of a[i][k] * b[k][j] (O(n^3)).
>
> **Example:** Transpose a table of products-by-month into months-by-product.

```
static int[][] transpose(int[][] m) {
  int[][] t = new int[m[0].length][m.length];
  for (int i = 0; i < m.length; i++)
    for (int j = 0; j < m[0].length; j++) t[j][i] = m[i][j];
  return t;
}
static int[][] multiply(int[][] a, int[][] b) {
  int[][] r = new int[a.length][b[0].length];
  for (int i = 0; i < a.length; i++)
    for (int j = 0; j < b[0].length; j++)
      for (int k = 0; k < b.length; k++) r[i][j] += a[i][k] * b[k][j];
  return r;
}
```

Output & Concept Trick Questions (2)

134. What does 1.0 / 0.0 return? Will it throw an exception or a compile-time error?

> **Answer:** It returns Infinity: floating-point division by zero does not throw and compiles fine. 0.0/0.0 gives NaN. Only integer division (1/0) throws ArithmeticException.
>
> **Example:** Dividing total by zero items in an average-price calculation: double gives Infinity, int throws.

```
System.out.println(1.0 / 0.0);   // Infinity
System.out.println(0.0 / 0.0);   // NaN
// System.out.println(1 / 0);    // ArithmeticException
```

135. Can you make an array volatile? Can you pass a negative number as an array size? **[asked x2 in your bank]**

> **Answer:** volatile applies only to the array reference, not its elements, so an array cannot be made volatile element-wise. A negative size compiles but throws NegativeArraySizeException at runtime.
>
> **Example:** A negative quantity used to size an array of items fails at runtime.

```
static volatile int[] arr = new int[5];   // only the reference is volatile
static void bad() {
  int[] x = new int[-1];                    // NegativeArraySizeException at runtime
}
```

Section 4: Low Priority / Quick Revision (16 questions)

Rarely asked for automation testers. Skim the idea, do not spend prep time here before Sections 1 to 3.

String Programs (4)

136. Convert a string to title case.

> **Answer:** Split into words, upper-case the first letter and lower-case the rest of each word.
>
> **Example:** Format product names as "Blue Cotton Shirt" for display checks.

```
static String titleCase(String s) {
  StringBuilder sb = new StringBuilder();
  for (String w : s.split(" "))
    sb.append(Character.toUpperCase(w.charAt(0)))
      .append(w.substring(1).toLowerCase()).append(" ");
  return sb.toString().trim();
}
```

137. Convert a string to a character array.

> **Answer:** Use s.toCharArray().
>
> **Example:** Loop over the characters of an entered coupon code.

```
char[] chars = "SAVE10".toCharArray();
```

138. Replace spaces with %20 (URL encoding).

> **Answer:** Replace each space with %20, as done in URL encoding (replace(" ", "%20")).
>
> **Example:** Build a search URL for a keyword containing spaces.

```
static String encode(String s) {
  return s.replace(" ", "%20");   // "red shoes" -> "red%20shoes"
}
```

139. Convert a sentence into an acronym.

> **Answer:** Take the first letter of every word and upper-case it.
>
> **Example:** Create a short code like "BCT" for "Blue Cotton T-shirt".

```
static String acronym(String s) {
  StringBuilder sb = new StringBuilder();
  for (String w : s.split("\\s+")) sb.append(Character.toUpperCase(w.charAt(0)));
  return sb.toString();
}
```

Number & Math Programs (5)

140. Find the Roman equivalent of a decimal number.

> **Answer:** Keep arrays of values and symbols in descending order and subtract greedily.
>
> **Example:** Show a rating or edition number in Roman numerals.

```
static String roman(int n) {
  int[] v = {1000, 900, 500, 400, 100, 90, 50, 40, 10, 9, 5, 4, 1};
  String[] s = {"M","CM","D","CD","C","XC","L","XL","X","IX","V","IV","I"};
  StringBuilder sb = new StringBuilder();
  for (int i = 0; i < v.length; i++) while (n >= v[i]) { sb.append(s[i]); n -= v[i]; }
  return sb.toString();
}
```

141. Reverse a number and add until you get a palindrome.

> **Answer:** Repeat: reverse the number and add it, until the sum is a palindrome (use long and a step limit).
>
> **Example:** Number-puzzle style logic; rarely tied to a real test.

```
static long reverseAdd(long n) {
  for (int i = 0; i < 100; i++) {
    long rev = Long.parseLong(new StringBuilder(String.valueOf(n)).reverse().toString());
    if (rev == n) return n;
    n += rev;
  }
  return -1;
}
```

142. Find the largest number less than a given number without a given digit.

> **Answer:** Count down from n - 1 and return the first number whose text does not contain the digit.
>
> **Example:** Pick the nearest lower order number without a banned digit.

```
static int largestWithout(int n, char digit) {
  for (int i = n - 1; i > 0; i--)
    if (String.valueOf(i).indexOf(digit) < 0) return i;
  return -1;
}
```

143. Find the square root of a number.

> **Answer:** Use Math.sqrt(n), or Newton's method for a manual calculation.
>
> **Example:** Calculate a distance or tolerance value in a formula check.

```
static double sqrt(double n) {
  double x = n;
  for (int i = 0; i < 20; i++) x = (x + n / x) / 2;   // Newton's method
  return x;                                            // or Math.sqrt(n)
}
```

144. Find trigonometric values of an angle in Java.

> **Answer:** Convert degrees to radians, then use Math.sin, Math.cos and Math.tan.
>
> **Example:** Rarely needed; used only in calculation formulas.

```
static void trig(double deg) {
  double rad = Math.toRadians(deg);
  System.out.println(Math.sin(rad) + " " + Math.cos(rad) + " " + Math.tan(rad));
}
```

Array Programs (4)

145. Find the minimum value in a rotated sorted array.

> **Answer:** Binary search: compare mid with the right end to decide which half contains the minimum (O(log n)).
>
> **Example:** Find the lowest value in a rotated price list.

```
static int minRotated(int[] a) {
  int lo = 0, hi = a.length - 1;
  while (lo < hi) {
    int mid = (lo + hi) >>> 1;
    if (a[mid] > a[hi]) lo = mid + 1; else hi = mid;
  }
  return a[lo];
}
```

146. Find all elements that appear more than n/k times.

> **Answer:** Count frequencies with a HashMap and return the elements whose count exceeds n/k (O(n)).
>
> **Example:** Find products that appear in more than a quarter of all orders.

```
static List<Integer> moreThan(int[] a, int k) {
  Map<Integer, Integer> m = new HashMap<>();
  for (int x : a) m.merge(x, 1, Integer::sum);
  List<Integer> out = new ArrayList<>();
  for (Map.Entry<Integer, Integer> e : m.entrySet())
    if (e.getValue() > a.length / k) out.add(e.getKey());
  return out;
}
```

147. Find the smallest positive integer that cannot be represented as a subset sum.

> **Answer:** For a sorted array keep res = 1; if the next element is <= res add it, otherwise res is the answer (O(n)).
>
> **Example:** Find the smallest amount that cannot be paid with the available coins.

```
static int smallestNonSum(int[] sorted) {
  int res = 1;
  for (int x : sorted) { if (x > res) break; res += x; }
  return res;
}
```

148. Find the minimum number of railway platforms required.

> **Answer:** Sort arrival and departure times; sweep with two pointers and track the maximum overlap (O(n log n)).
>
> **Example:** Find the peak number of simultaneous delivery slots.

```
static int platforms(int[] arr, int[] dep) {
  Arrays.sort(arr); Arrays.sort(dep);
  int need = 0, max = 0, i = 0, j = 0;
  while (i < arr.length) {
    if (arr[i] <= dep[j]) { need++; i++; } else { need--; j++; }
    max = Math.max(max, need);
  }
  return max;
}
```

Sorting, Searching & Complexity (1)

149. Implement Bucket sort and Counting sort. **[asked x2 in your bank]**

> **Answer:** Counting sort counts occurrences of each value and rebuilds the array (O(n + k)). Bucket sort distributes values into ranges, sorts each bucket and joins them.
>
> **Example:** Sort ratings (1 to 5) quickly using counting sort.

```
static void countingSort(int[] a, int max) {
  int[] count = new int[max + 1];
  for (int x : a) count[x]++;
  int k = 0;
  for (int v = 0; v <= max; v++) while (count[v]-- > 0) a[k++] = v;
}
```

Linked List, Stack, Matrix & Advanced Problems (2)

150. Print a Spiral Matrix.

> **Answer:** Print the top row, right column, bottom row and left column, then shrink the boundaries.
>
> **Example:** Traverse a grid of product tiles in spiral order.

```
static void spiral(int[][] m) {
  int top = 0, bot = m.length - 1, left = 0, right = m[0].length - 1;
  while (top <= bot && left <= right) {
    for (int j = left; j <= right; j++) System.out.print(m[top][j] + " ");
    top++;
    for (int i = top; i <= bot; i++) System.out.print(m[i][right] + " ");
    right--;
    if (top <= bot) {
      for (int j = right; j >= left; j--) System.out.print(m[bot][j] + " ");
      bot--;
    }
    if (left <= right) {
      for (int i = bot; i >= top; i--) System.out.print(m[i][left] + " ");
      left++;
    }
  }
}
```

151. Find shortest-path weights from a source vertex to all vertices in a graph.

> **Answer:** Dijkstra's algorithm: use a min priority queue and relax edges to find the shortest distance from the source to every vertex (O(E log V)).
>
> **Example:** Find the cheapest shipping route between warehouses.

```
static int[] dijkstra(List<int[]>[] g, int src) {   // edge = {to, weight}
  int[] dist = new int[g.length];
  Arrays.fill(dist, Integer.MAX_VALUE); dist[src] = 0;
  PriorityQueue<int[]> pq = new PriorityQueue<>((x, y) -> x[1] - y[1]);
  pq.add(new int[]{src, 0});
  while (!pq.isEmpty()) {
    int[] cur = pq.poll();
    if (cur[1] > dist[cur[0]]) continue;
    for (int[] e : g[cur[0]]) {
      int nd = cur[1] + e[1];
      if (nd < dist[e[0]]) { dist[e[0]] = nd; pq.add(new int[]{e[0], nd}); }
    }
  }
  return dist;
}
```